

Project Report - DLGEN AI

Name: Venkata Ganapathi Subramanian V

Roll Number: 23F1000054

Course: BSDA2001P - Introduction to Deep Learning and Generative AI

Project Title: Smart MCQ Solver

Date: July 2026

Abstract / Executive Summary:

Smart MCQ Solver is an intelligent multiple-choice question and answer system, which was developed as final project for BSDA2001P – Introduction to Deep Learning and Generative AI Course. The goal is to guess the top three possible correct answers out of five possible answers (A–E) for a question. Each question/option pair is defined as either right or wrong and the best 3 results are reported for assessment.

We experimented with various machine learning/Deep Learning models such as TF-IDF combined with Logistic Regression, MLP, BiLSTM and fine-tuned DeBERTa models. The model with the most favorable results was DeBERTa-v3-base, with a validation F1-score of 0.9208 and an accuracy of 97.0%. The final solution has been deployed with Gradio on Hugging Face Spaces, the trained model is on the Hugging Face Model Hub, and the full code is available on GitHub.

Introduction:

Problem Statement

Multiple-choice question answering (MCQA) would have a system to analyze a question as well as five candidate answers, and determine the most suitable answers. In this project, the task will be defined as a binary relevance classification problem where each question–option pair is considered as a binary classification problem, having only two possibilities: correct or incorrect. For inference, all the 5 options are scored and sorted and the highest 3 inferences are returned back to the output. The dataset is comprised of science related questions whose answer choices are mostly semantically similar, making it well suited to transformer based language models which have the ability to capture context.

Project Objectives

- Exploratory Data Analysis (EDA) to gain understanding of the structure of the data, the answer distribution, and characteristics of text.
- Describe and compare various machine learning and deep learning models to be used in prediction of MCQ task.
- Convert a multi-choice question with 5 options problem to a binary classification problem with question–option pairs.
- Reproduce and compare track experiments with Weights & Biases.
- Optimize pre-trained DeBERTa-v3 model and determine optimum model architecture.
- Publish the final model as a web app in Hugging Face Spaces.
- Create fully functional submissions for competition and compare the levels of performance of various models.

Report Overview

This is the entire development of Smart MCQ Solver project. It includes database and pre-processing, the architecture of the models and experimentation, performance evaluation, deployment, challenges faced in implementation, possible future enhancements, and finally some conclusions obtained in the project.

3. Dataset and Preprocessing

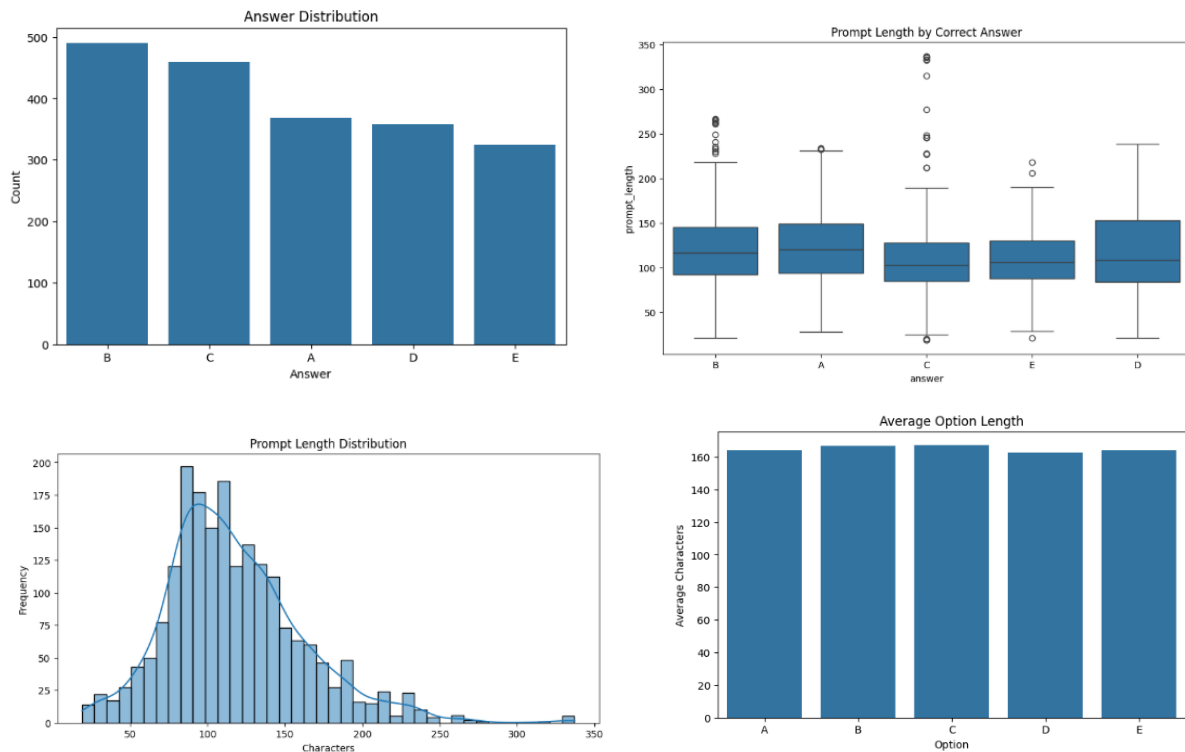
Dataset Description

The dataset used in this project is from the Smart MCQ Solver Challenge. It consists of 2,000 training samples and 500 test samples. The training instances include a question prompt, 5 answer options (A – E) and the correct answer label, while the test set only includes the question and the answer options. The goal is to guess the three top choices possible for every question.

Exploratory Data Analysis (EDA)

The characteristics of the data were explored prior to model construction. There were no missing data in the data set and no data imputation was needed. There were 242 duplicate prompts; this resulted in 1,758 unique questions. Prompt average length was about 118

characters and the answer options averaged 163-167 characters. The distribution of answer labels was a moderate class balance with the highest number of B and lowest number of E.



Answer Distribution: The answer labels are moderately imbalanced, with option **B** appearing most frequently and option **E** the least, but all five classes are reasonably represented.

Prompt Length Distribution: Most question prompts are between **80 and 140 characters**, with an average length of approximately **118 characters**, indicating relatively consistent prompt sizes.

Average Option Length: The average lengths of answer options **A–E** are nearly identical (approximately **163–167 characters**), showing a consistent option formatting throughout the dataset.

Prompt Length by Correct Answer: Prompt lengths are similarly distributed across all answer labels, suggesting that prompt length alone does not strongly influence the correct answer category.

Primary Tokenization Strategy

Different tokenization strategies were used based on the model architecture. The TF-IDF baseline used **TfidfVectorizer**, while the MLP and BiLSTM models used **MiniLM sentence embeddings**. For the best-performing models, **DeBERTa-v3-small** and **DeBERTa-v3-base**, the **Hugging Face AutoTokenizer** was used to encode each question and answer option as a sentence pair.

Justification

Pair-wise tokenization enables the DeBERTa models to capture the contextual relationship between a question and its candidate answer. This provides richer semantic representations than traditional feature extraction methods and significantly improves prediction performance.

Implementation Details

The tokenizer was implemented using the Hugging Face **AutoTokenizer** library with padding and truncation to generate fixed-length input sequences. The resulting token IDs and attention masks were used to fine-tune the DeBERTa sequence classification models.

4. Modeling & Experimentation

For evaluation five models were implemented and compared the various approaches to the Smart MCQ Solver task. The same preprocessing procedure and binary classification model either correct or incorrect was used for all models with each question–option combination being labeled. An 80:20 stratified train–validation split was used for evaluation, resulting in 8,000 training samples and 2,000 validation samples. All training was done on Google Colab and model checkpoints and experiment logs were stored on Google Drive and Weights & Biases.

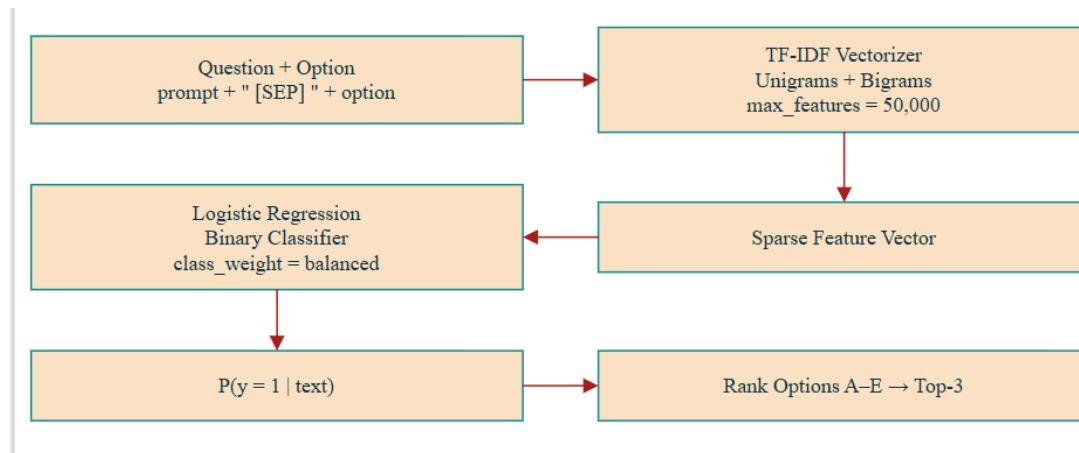
Logistic regression with TF-IDF (baseline) :

Architecture: This is a baseline model that uses a TF-IDF vectorizer and a Logistic Regression classifier. The question – set of options are sparsely encoded with textual features before binary classification.

Salient Points: This model was chosen due to being an extensively used classical NLP baseline that is simple, computationally efficient and interpretable. It is not as powerful in learning deep semantic relationships between questions and answer options as other deep

learning models, but it offers a sound benchmark for evaluating those more powerful models.

Training Strategy: To overcome the class imbalance problem, the model was trained with unigram and bigram features extracted from TF-IDF and a balanced Logistic Regression as the classifier.

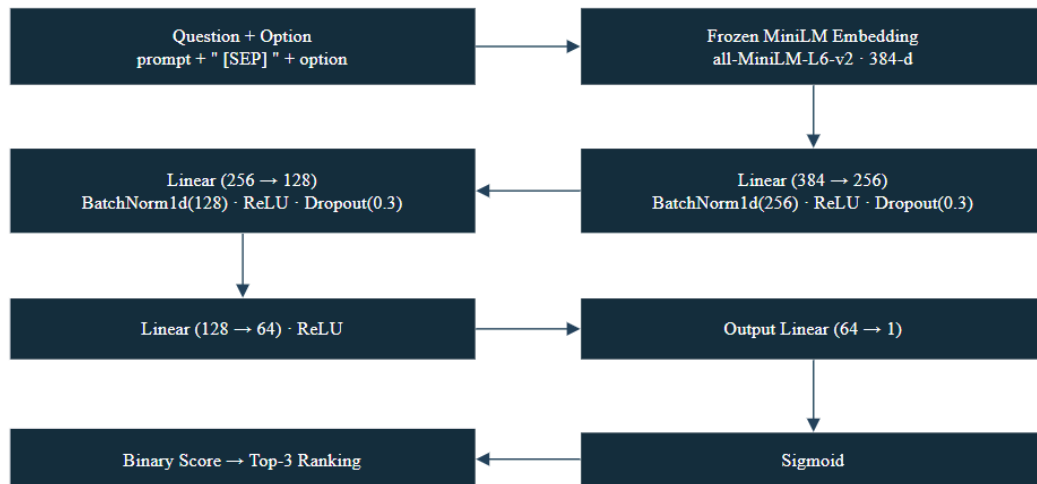


Multi-Layer Perceptron (MLP)

Architecture: In order to train the Model, 384-dimensional sentence embeddings obtained from the MiniLM were applied to an MLP. It is a multi-layers fully connected ReLU network, which was regularized by dropout and batch normalization.

Salient Points: The model is able to learn deeper semantic relationships than just keywords, and is still quite affordable to run with MiniLM embeddings. While the MLP offers richer representations of features, it has limitations due to the use of frozen embeddings, compared to the TF-IDF baseline.

Training Strategy: This model was trained in PyTorch with the AdamW optimizer and Binary Cross-Entropy loss function for 10 epochs, and recorded results with Weights & Biases.

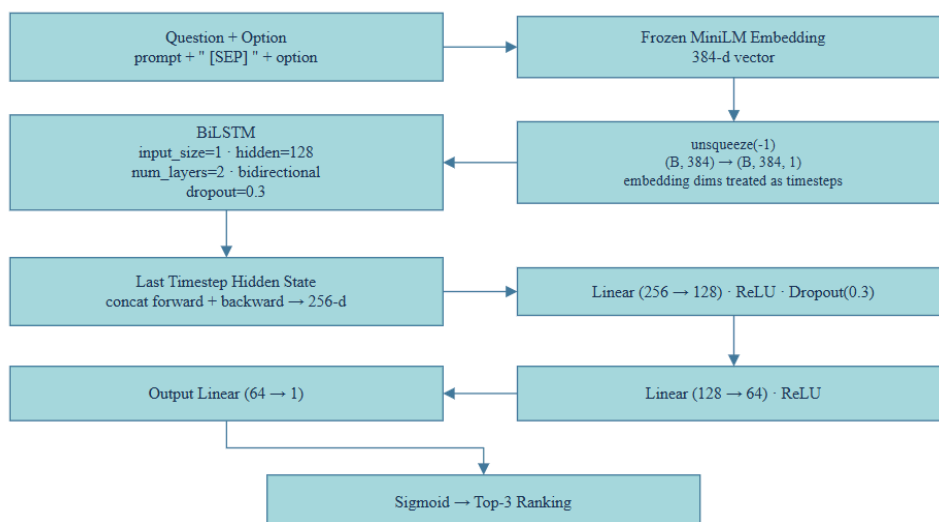


Bidirectional LSTM (BiLSTM):

Architecture: The BiLSTM model takes as input the same MiniLM sentence embeddings as used in input, and passes through a 2-layer bidirectional Long Short-Term Memory Network and fully connected classification layers.

Salient Points: Bidirectional architecture was devised to encode sequential dependencies in the embedding representation. The model was not able to find meaningful patterns of this task and was not competitive to the other methods.

Training Strategy: Hyperparameters were used in training as in the MLP: AdamW optimizer, Binary Cross-Entropy loss and 10 training epochs.



DeBERTa-v3-small

Architecture: The first model using a transformer was DeBERTa-v3-small, which was trained on a binary sequence classification head. The tokenizer used for DeBERTa was used to encode each question and answer into paired inputs.

Salient Points: Unlike the regular transformer models, Salient Points of DeBERTa model can be identified by the attention disentangled approach that leads to a better understanding of the context. The small variant is a good compromise between speed and success in the prediction.

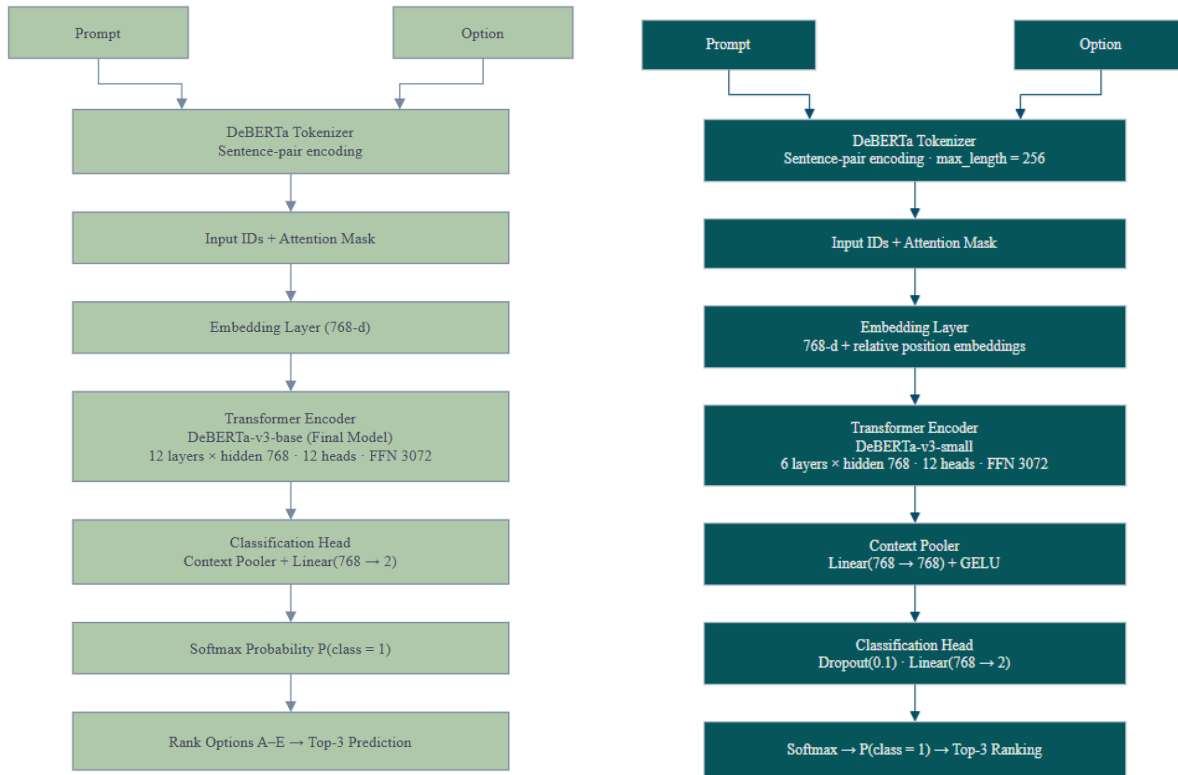
Training Strategy: Fine-tuning was conducted via the Hugging Face Trainer API, AdamW optimizer, scheduling a learning rate, and selecting the best model from the validation dataset.

DeBERTa-v3-base (Final Model).

Architecture: The model could be either DeBERTa-v3-base or DeBERTa-v3-large, both with a binary sequence classification head. Just as with the small variant, there are pairs of questions/candidate answers and their relevance is determined.

Salient Points: This model had the largest validation accuracy and F1 score of all the models implemented. As for architecture, it is the largest, thus facilitating the understanding of the semantic relationships, and is the most performing model in the project. The trained model was chosen to be deployed to the Hugging Face Model Hub and Hugging Face Spaces.

Training Strategy: The model was fine-tuned with the Hugging Face Trainer, AdamW optimizer, linear learning rate scheduling, gradient accumulation and model checkpointing to achieve the best validation results.



Replaced the input with a paired question–option, the transformer encoder, and the sequence classification head in the DeBERTa-v3-base architecture. Replaced the question–option input, the transformer encoder, and the sequence classification head with the DeBERTa-v3-small architecture.

Evaluation Metrics

The performance of all models was evaluated on the binary validation dataset using **Accuracy** and **F1-score**. Accuracy measures the proportion of correctly classified question–option pairs, while the F1-score provides a balanced evaluation of precision and recall for the positive class. During inference, the probability scores of all five answer options are ranked, and the top three options are returned as the final prediction.

Training Performance

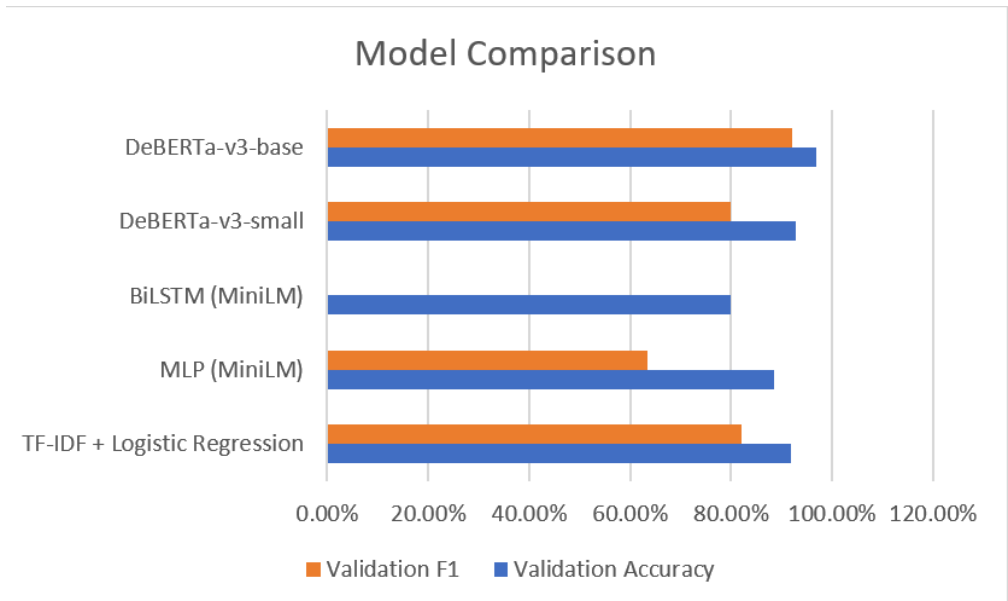
Training progress was monitored using **Weights & Biases (W&B)**. The transformer-based DeBERTa models showed steady convergence, with training loss decreasing consistently over successive epochs. The MLP model gradually improved after the initial epochs, whereas the BiLSTM model failed to converge and remained

close to the majority-class baseline. The TF-IDF baseline achieved competitive performance with minimal training time.

Comparative Analysis

The performance of all implemented models is summarized below. Classical machine learning approaches provided strong baseline results, while transformer-based models significantly improved classification accuracy by capturing richer semantic relationships between questions and answer options.

Model	Validation Accuracy	Validation F1	Training Time
TF-IDF + Logistic Regression	91.95%	0.8209	Instant
MLP (MiniLM)	88.45%	0.6351	10 Epochs
BiLSTM (MiniLM)	80.00%	0.0000	10 Epochs
DeBERTa-v3-small	92.95%	0.7983	~8 Minutes
DeBERTa-v3-base	97.00%	0.9208	~23 Minutes



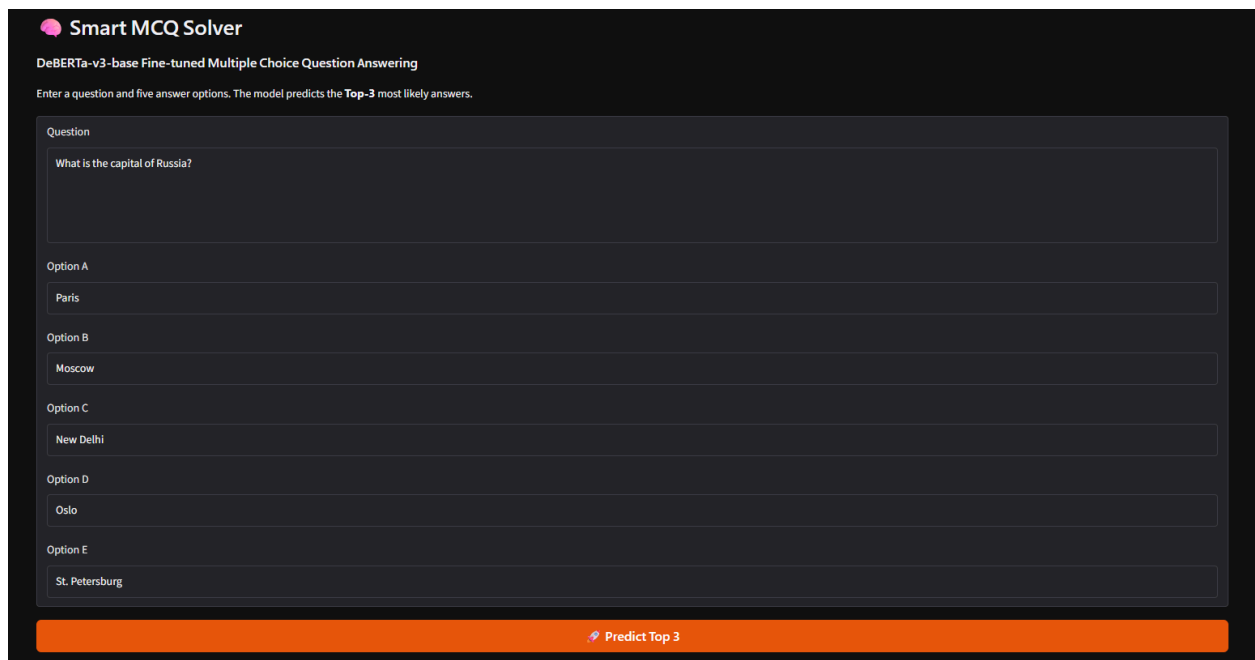
Best Performing Model

Among all evaluated approaches, **DeBERTa-v3-base** achieved the highest validation accuracy (**97.0%**) and F1-score (**0.9208**). Its ability to model contextual relationships between questions and answer options enabled superior performance over both classical machine learning and custom neural network models. The trained model was selected as the final deployment model and published on the Hugging Face Model Hub.

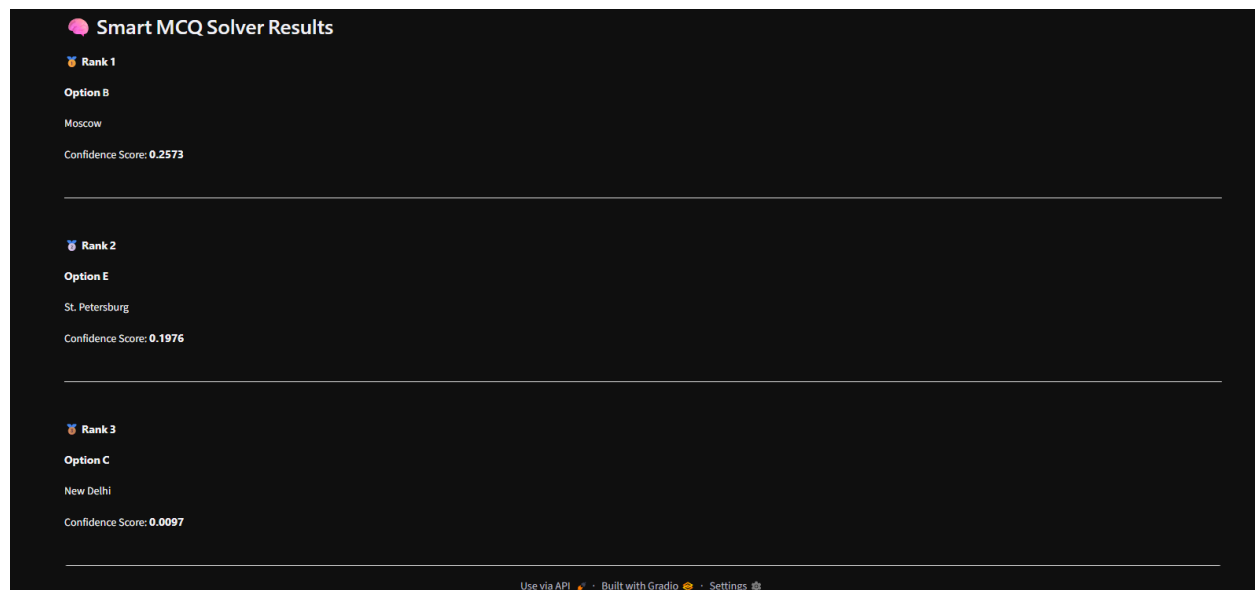
Deployment

To show the usage of the project in practice, the last model, DeBERTa-v3-base, was deployed as an interactive web application, built with Gradio and put on Hugging Face Spaces. Users can enter a question along with its five answer options, and the system predicts the top three most likely correct answers with their confidence scores.

It loads the trained model and tokenizer from the Hugging Face Model Hub and infers for each question–option pair, sorts the probabilities and returns the top three options. The whole project, the source code, the notebooks, trained models and deployment files are kept on GitHub.



The screenshot shows a web application titled "Smart MCQ Solver" with a subtitle "DeBERTa-v3-base Fine-tuned Multiple Choice Question Answering". Below the subtitle, a instruction reads: "Enter a question and five answer options. The model predicts the **Top-3** most likely answers." The interface contains five input fields for options, labeled "Option A" through "Option E". The "Question" field contains the text "What is the capital of Russia?". The "Option A" field contains "Paris", "Option B" contains "Moscow", "Option C" contains "New Delhi", "Option D" contains "Oslo", and "Option E" contains "St. Petersburg". At the bottom of the interface is a large orange button labeled "Predict Top 3".



Challenges Faced

Imbalanced Data: All the questions were transformed into five binary samples, which resulted in an imbalanced dataset and led to some models making a starting guess for the majority class. This had a significant impact on the first training run, specifically the F1-score.

BiLSTM Performance: The BiLSTM model yielded no meaningful representation from fixed embeddings of MiniLM, and failed to converge after multiple epochs.

To fine-tune the DeBERTa-v3-base model, GPU resources were needed and it also took a significantly longer time in training, which restricted the number of experiments able to be conducted.

Package Compatibility: Some dependency conflicts were identified when using Transformers, PyTorch and Hugging Face Spaces, and had to be resolved during development and deployment.

Duplicate Question prompts: Question prompts were present in the dataset multiple times and might lead to data leakage if these were not properly split when training and validating.

General problems: Success in deploying the Hugging Face Space involved resolving dependency and environment-related problems.

Future Work

While the Results of the proposed Smart MCQ Solver proved to be quite satisfactory, there are a number of areas for further improvement:

- Try out bigger or newer language models like DeBERTa-v3-large, ModernBERT or even Qwen to further boost prediction accuracy.
- Class Imbalance Handling: Explore ways of better addressing the binary class imbalance, including focal loss, weighted loss functions, and oversampling.
- Model Ensembling: Fuse models like TF-IDF and DeBERTa to obtain more robust predictions and better Top-3 ranking results.
- Cross-Validation: to get more reliable performance estimates and better generalization ability, employ k-fold instead of a train/valid split.
- Enhanced Deployment: Add batch prediction, explanation of model predictions and add support for other question types apart from science-based MCQs.

Conclusion

The project implemented an intelligent multiple-choice question answering system with both traditional machine learning and transformer-based deep learning methods, achieving successful results. There were 5 models implemented and evaluated, including a baseline TF-IDF model to fine-tuned DeBERTa-based transformers. After testing all the models, the best performance was obtained by DeBERTa-v3-base with a validation accuracy of 97.0% and F1-score of 0.9208, which will be applied to the deployment.

The project provided valuable practical experience in data preprocessing, model development, experiment tracking, performance evaluation, and model deployment using Hugging Face and Gradio. In total, this Smart MCQ Solution outlines an end-to-end deep learning solution: exploratory data analysis, model training, and deployment of the model as an interactive web application.

References

1. He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention*. International Conference on Learning Representations (ICLR).

2. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP.
3. Wolf, T., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP: System Demonstrations.
4. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS).
5. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

Important Links

- **GitHub Repository:** <https://github.com/venkata0803/Smart-MCQ-Solver>
- **Hugging Face Model:**
<https://huggingface.co/vvenkata/smart-mcq-solver-deberta-base>
- **Hugging Face Space:** <https://huggingface.co/spaces/vvenkata/smart-mcq-solver>
- **Kaggle Competition:**
<https://www.kaggle.com/competitions/smart-mcq-solver-challenge>
- **Weights & Biases Project:**
<https://wandb.ai/23f1000054-dl-genai-project/23f1000054-t22026>